

CI-CD-Pipeline-Jenkins-Docker

CI/CD Pipeline for Java Web Application using Jenkins and Docker

1. Project Overview

The **CI-CD-Pipeline-Jenkins-Docker** project demonstrates the implementation of an automated Continuous Integration and Continuous Deployment (CI/CD) pipeline for a Java-based Online Bookstore web application. The objective of this project is to automate the software delivery process—from source code integration to application deployment—using industry-standard DevOps tools such as GitHub, Jenkins, Maven, Docker, and Apache Tomcat.

The pipeline minimizes manual intervention, ensures consistent deployments, and enables rapid delivery of application updates.

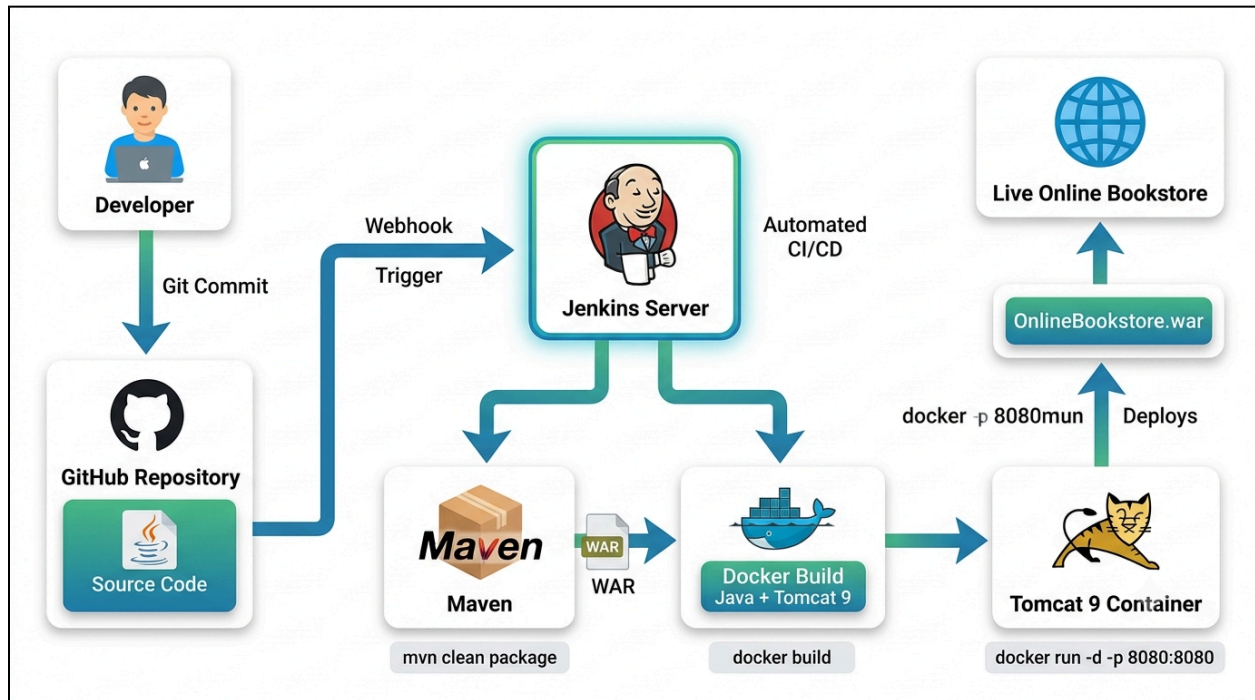
2. Objectives

- Automate the software build process.
- Reduce manual deployment effort.
- Improve software delivery speed.
- Ensure reliable and repeatable deployments.
- Containerize the application for portability.
- Demonstrate a complete CI/CD workflow using Jenkins and Docker.

3. Technologies Used

This project utilizes several industry-standard technologies to implement an efficient CI/CD pipeline. Java is used for developing the Online Bookstore web application. Apache Maven handles build automation and dependency management, ensuring that the application is compiled and packaged consistently. Git is used for version control, enabling developers to track and manage code changes effectively. GitHub serves as the remote source code repository and provides webhook integration with Jenkins. Jenkins acts as the core Continuous Integration and Continuous Deployment tool, automating the build, packaging, and deployment processes. Docker is used to containerize the application and its deployment environment, making deployments portable and consistent across systems. Apache Tomcat 9 hosts the packaged WAR application inside a containerized server environment. Finally, Linux provides the deployment environment on which Jenkins, Docker, and Tomcat are configured and executed.

4. System Architecture



5. Workflow

Step 1: Code Development

The developer writes or modifies the Java source code for the Online Bookstore application using an IDE such as Eclipse or IntelliJ IDEA.

After completing the changes, the code is committed and pushed to the GitHub repository.

Step 2: GitHub Repository

GitHub serves as the centralized version control system.

Whenever new code is pushed, a GitHub Webhook automatically sends a notification to Jenkins.

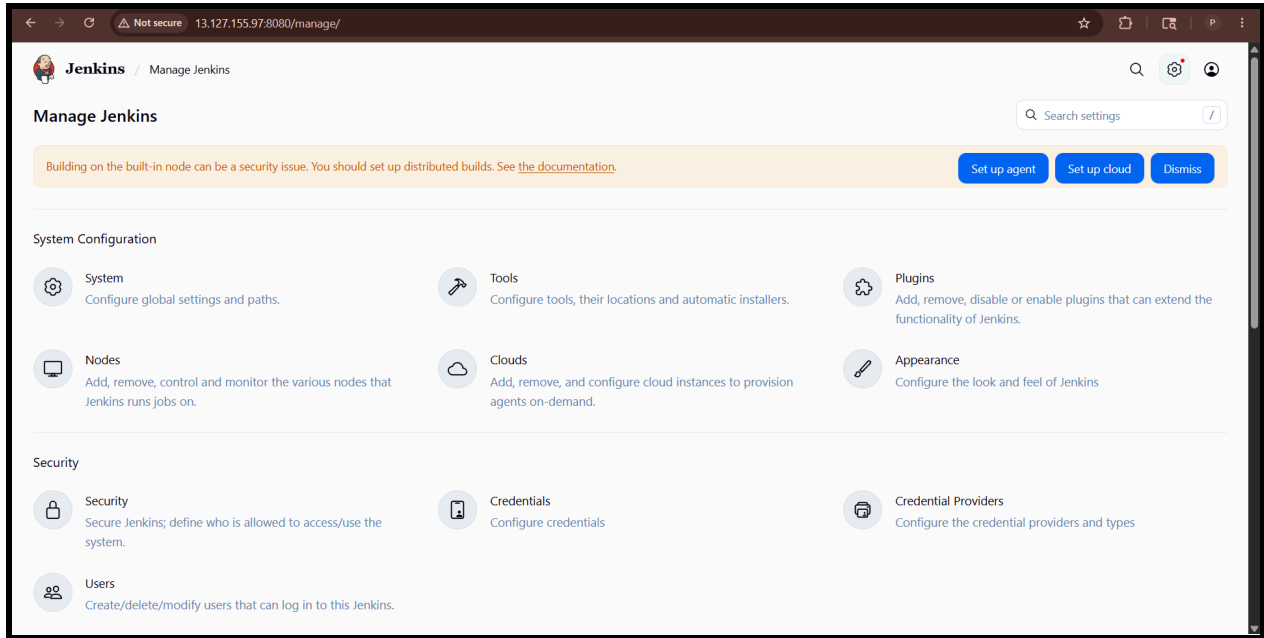
This eliminates the need for manually starting the build process.

Step 3: Jenkins Pipeline

Jenkins receives the webhook request and starts the CI/CD pipeline automatically.

The Jenkins pipeline performs the following tasks:

- Pull latest source code
- Install project dependencies
- Compile Java code
- Execute Maven build
- Generate WAR artifact
- Build Docker image
- Deploy Docker container



Step 4: Maven Build

Jenkins executes:

mvn clean package

```
apache-maven-3.9.15-bin.tar.gz      100%[=====]
2026-04-27 04:04:27 (151 MB/s) - 'apache-maven-3.9.15-bin.tar.gz' saved [9236330/9236330]

[root@ip-172-31-35-184 opt]# ls
apache-maven-3.9.15-bin.tar.gz  aws
[root@ip-172-31-35-184 opt]# tar -xvf apache-maven-3.9.15-bin.tar.gz
apache-maven-3.9.15/README.txt
```

The Maven lifecycle performs:

- Cleaning previous builds
- Downloading required dependencies
- Compiling source code
- Running tests (if configured)
- Packaging the application

Output:

OnlineBookstore.war

6. Methodology

The project follows a Continuous Integration and Continuous Deployment approach.

Phase 1 – Source Code Management

- Developer modifies code.
- Changes are committed locally.
- Code is pushed to GitHub.

Phase 2 – Continuous Integration

- GitHub webhook triggers Jenkins.
- Jenkins fetches the latest source code.
- Maven compiles the application.
- WAR package is generated.

Phase 3 – Continuous Deployment

- Jenkins builds a Docker image.
- Docker image includes Tomcat and WAR file.
- Jenkins starts the Docker container.
- Application becomes available automatically.

7. Project Features

- Automated build process
- Automated deployment
- GitHub integration
- Jenkins pipeline automation
- Docker containerization
- Maven dependency management
- Portable deployment environment
- Faster release cycle
- Reduced manual errors
- Easy scalability

8. Advantages

- Fully automated deployment
- Faster software delivery
- Improved development workflow
- Consistent build environment
- Easy rollback through Docker images
- Reduced deployment failures
- Better collaboration among developers
- Platform-independent deployment

9. Jenkins Pipeline Summary

1. Developer pushes code to GitHub.
2. GitHub webhook triggers Jenkins.
3. Jenkins pulls the latest repository.
4. Maven builds the project.
5. WAR artifact is generated.
6. Docker image is created.
7. Docker container is launched.
8. Tomcat deploys the application.
9. Users access the live Online Bookstore application.

10. Project Use Cases

This project can be used for:

- Learning DevOps concepts
- Understanding CI/CD pipelines
- Java web application deployment
- Docker container deployment
- Jenkins automation practice
- College mini and major projects
- Resume and portfolio demonstration
- Internship and placement preparation

11. Future Enhancements

The project can be extended with advanced DevOps practices such as:

- Automated unit testing using JUnit
- Code quality analysis using SonarQube
- Artifact management with Nexus Repository
- Container orchestration using Kubernetes
- Infrastructure as Code using Terraform
- Monitoring using Prometheus and Grafana
- Continuous Delivery using ArgoCD
- Cloud deployment on AWS, Azure, or Google Cloud Platform

12. Conclusion

The **CI-CD-Pipeline-Jenkins-Docker** project successfully demonstrates the implementation of an automated DevOps pipeline for a Java-based Online Bookstore application. By integrating GitHub, Jenkins, Maven, Docker, and Apache Tomcat, the project automates the complete software delivery lifecycle—from source code management to application deployment.

The use of Jenkins enables continuous integration by automatically building and packaging the application whenever code changes are pushed to GitHub. Docker provides a consistent and portable deployment environment, while Tomcat hosts the packaged WAR application inside a container.

This project showcases essential DevOps practices including build automation, continuous integration, containerization, and automated deployment. It significantly reduces manual effort, improves deployment consistency, and provides a scalable foundation that can be further enhanced with tools such as SonarQube, Kubernetes, and cloud platforms.

Overall, the project serves as a practical implementation of a modern CI/CD pipeline and demonstrates the importance of automation in accelerating software delivery while maintaining reliability and deployment quality.